

Pseudocode Writing Task - Answers

Question 1 - Throughput, Packet Loss, and Classification

Objective: Calculate throughput and packet loss, validate the inputs, classify performance, and print a complete summary.

Given values: Total data received = 8000 bits;

- time = 4 seconds;
- packets sent = 200;
- packets received = 180.

```
ALGORITHM EvaluateNetworkPerformance
  CONSTANT THROUGHPUT_LIMIT <- 1500
  CONSTANT LOSS_LIMIT <- 10

  INPUT totalDataBits, timeSeconds, packetsSent, packetsReceived

  IF totalDataBits < 0 OR timeSeconds < 0 OR packetsSent < 0 OR packetsReceived < 0 THEN
    DISPLAY "Error: Negative input values are not allowed."
    STOP
  END IF

  IF timeSeconds = 0 THEN
    DISPLAY "Error: Time must be greater than zero."
    STOP
  END IF

  IF packetsSent = 0 THEN
    DISPLAY "Error: Total packets sent must be greater than zero."
    STOP
  END IF

  IF packetsReceived > packetsSent THEN
    DISPLAY "Error: Packets received cannot exceed packets sent."
    STOP
  END IF

  throughputBps <- totalDataBits / timeSeconds
  lostPackets <- packetsSent - packetsReceived
  packetLossPercent <- (lostPackets / packetsSent) * 100
```

```
// PSEUDOCODE CONTINUED
```

```
IF throughputBps > THROUGHPUT_LIMIT AND packetLossPercent < LOSS_LIMIT THEN  
    classification <- "Efficient"  
ELSE  
    classification <- "Inefficient"  
END IF
```

```
DISPLAY "----- NETWORK PERFORMANCE SUMMARY -----"  
DISPLAY "Data received    : ", totalDataBits, " bits"  
DISPLAY "Observation time : ", timeSeconds, " seconds"  
DISPLAY "Packets sent     : ", packetsSent  
DISPLAY "Packets received  : ", packetsReceived  
DISPLAY "Packets lost      : ", lostPackets  
DISPLAY "Throughput        : ", throughputBps, " bps"  
DISPLAY "Packet loss       : ", packetLossPercent, "%"  
DISPLAY "Classification    : ", classification  
END ALGORITHM
```

Calculated Result

Metric	Calculation / Result	Interpretation
Throughput	$8000 / 4 = 2000$ bps	Greater than 1500 bps
Packets lost	$200 - 180 = 20$	20 packets were not received
Packet loss	$(20 / 200) \times 100 = 10\%$	Not less than 10%
Classification	Inefficient	Both efficiency conditions are not satisfied

Final answer: Throughput is 2000 bps and packet loss is 10%. The network is classified as Inefficient because the rule requires packet loss to be strictly less than 10%, not equal to 10%.

Question 2 - Continuous Bandwidth Utilization Monitoring

Objective: Periodically collect bandwidth usage for every branch link, store the readings, detect overload, and recommend a suitable backup link.

Data structures: Arrays hold link names, capacities, current usage, utilization, status, backup eligibility, and a time-stamped usage history for every link.

```

ALGORITHM MonitorBandwidthUtilization
  CONSTANT OVERLOAD_LIMIT <- 80
  CONSTANT WARNING_LIMIT <- 70
  CONSTANT MONITOR_INTERVAL <- 60 seconds

  INPUT linkCount
  IF linkCount <= 0 THEN
    DISPLAY "Error: At least one network link is required."
    STOP
  END IF

  DECLARE linkName[1..linkCount]
  DECLARE capacityMbps[1..linkCount]
  DECLARE usedMbps[1..linkCount]
  DECLARE utilization[1..linkCount]
  DECLARE linkStatus[1..linkCount]
  DECLARE backupEligible[1..linkCount]
  DECLARE usageHistory[1..linkCount] AS LIST

  READ configuration into linkName, capacityMbps, backupEligible
  monitoringEnabled <- TRUE

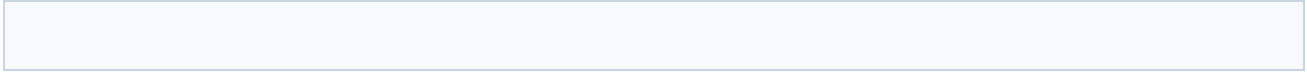
  WHILE monitoringEnabled = TRUE DO
    currentTime <- GET_CURRENT_TIME()

    // Phase 1: collect and store the latest reading for every link
    FOR i <- 1 TO linkCount DO
      linkStatus[i] <- GET_LINK_STATUS(linkName[i])

      IF capacityMbps[i] <= 0 THEN
        linkStatus[i] <- "INVALID"
        utilization[i] <- -1
        GENERATE_ALERT(linkName[i], "Invalid link capacity")
      ELSE IF linkStatus[i] <> "UP" THEN
        utilization[i] <- -1
        GENERATE_ALERT(linkName[i], "Link is unavailable")
      ELSE
        usedMbps[i] <- COLLECT_BANDWIDTH_USAGE(linkName[i])

        IF usedMbps[i] < 0 THEN
          utilization[i] <- -1
          GENERATE_ALERT(linkName[i], "Usage reading failed")
        ELSE
          utilization[i] <- (usedMbps[i] / capacityMbps[i]) * 100
          APPEND (currentTime, usedMbps[i], utilization[i])
            TO usageHistory[i]
        END IF
      END IF
    END FOR
  END FOR

```



```

// PSEUDOCODE CONTINUED

// Phase 2: analyze all current readings after collection
FOR i <- 1 TO linkCount DO
  IF linkStatus[i] = "UP" AND utilization[i] >= 0 THEN
    DISPLAY linkName[i], usedMbps[i], utilization[i], "%"

    IF utilization[i] > OVERLOAD_LIMIT THEN
      backup <- FIND_BEST_BACKUP(i, linkCount, linkStatus,
                                utilization, backupEligible)
      IF backup <> NONE THEN
        DISPLAY "OVERLOADED: Switch traffic from ", linkName[i],
              " to ", linkName[backup]
        GENERATE_ALERT(linkName[i], "Traffic diversion recommended")
      ELSE
        GENERATE_ALERT(linkName[i], "No safe backup link available")
      END IF
    ELSE IF utilization[i] >= WARNING_LIMIT THEN
      DISPLAY "WARNING: Link is approaching overload."
    ELSE
      DISPLAY "NORMAL: Link utilization is within limit."
    END IF
  END IF
END FOR

DISPLAY_CYCLE_SUMMARY(currentTime, utilization, linkStatus)
WAIT(MONITOR_INTERVAL)
END WHILE
END ALGORITHM

FUNCTION FIND_BEST_BACKUP(source, linkCount, status, utilization, eligible)
  best <- NONE
  lowestUtilization <- INFINITY

  FOR j <- 1 TO linkCount DO
    IF j <> source AND eligible[j] = TRUE AND status[j] = "UP"
      AND utilization[j] >= 0 AND utilization[j] <= 80
      AND utilization[j] < lowestUtilization THEN
      best <- j
      lowestUtilization <- utilization[j]
    END IF
  END FOR

  RETURN best
END FUNCTION

```

How the Algorithm Prevents Congestion

Real-time visibility: Every link is measured at the same interval, and the time-stamped history helps the administrator identify repeated peak-load periods.

Early action: A warning is produced from 70% utilization, while links above 80% are treated as overloaded before severe delay, queue build-up, or packet loss occurs.

Safe traffic distribution: The algorithm selects the available backup link with the lowest current utilization, preventing traffic from being shifted to another congested link.

Reliable operation: Invalid capacity, failed readings, unavailable links, and the absence of a safe backup are handled explicitly and reported through alerts.

Question 3 - Adaptive Best-Path Selection for Six Branches

Objective: Continuously choose the highest-quality path from headquarters to each of six branches using bandwidth, delay, and utilization rather than distance alone.

Path model: The network is stored as links and candidate paths. Each candidate path is a list of link IDs connecting headquarters to one branch.

Path Quality Formula

Component	Weight	Meaning
Bandwidth score	50%	Higher bottleneck bandwidth improves the score
Delay score	30%	Lower total path delay improves the score
Utilization score	20%	Lower average utilization improves the score

Formula: $\text{PathScore} = 100 \times (0.50 \times \text{BandwidthScore} + 0.30 \times \text{DelayScore} + 0.20 \times \text{UtilizationScore})$. Each component is normalized to the range 0 to 1. A path containing a failed or overloaded link is rejected before scoring.

```

ALGORITHM AdaptiveBestPathSelection
  CONSTANT BRANCH_COUNT <- 6
  CONSTANT OVERLOAD_LIMIT <- 80
  CONSTANT MONITOR_INTERVAL <- 60 seconds
  CONSTANT REFERENCE_BANDWIDTH <- configured maximum useful bandwidth
  CONSTANT ACCEPTABLE_DELAY <- configured maximum acceptable path delay

  INPUT linkCount, candidatePaths[1..BRANCH_COUNT]
  IF linkCount <= 0 OR REFERENCE_BANDWIDTH <= 0 OR ACCEPTABLE_DELAY <= 0 THEN
    DISPLAY "Error: Invalid network configuration."
    STOP
  END IF

  DECLARE bandwidth[1..linkCount]
  DECLARE delay[1..linkCount]
  DECLARE utilization[1..linkCount]
  DECLARE status[1..linkCount]
  DECLARE metricHistory[1..linkCount] AS LIST
  DECLARE currentPath[1..BRANCH_COUNT] <- NONE

  monitoringEnabled <- TRUE
  WHILE monitoringEnabled = TRUE DO
    currentTime <- GET_CURRENT_TIME()

    // Collect the latest metrics for every physical link
    FOR link <- 1 TO linkCount DO
      status[link] <- GET_LINK_STATUS(link)

      IF status[link] = "UP" THEN
        bandwidth[link] <- COLLECT_BANDWIDTH(link)
        delay[link] <- COLLECT_DELAY(link)
        utilization[link] <- COLLECT_UTILIZATION(link)

        IF bandwidth[link] <= 0 OR delay[link] < 0
          OR utilization[link] < 0 THEN
          status[link] <- "INVALID"
          GENERATE_ALERT(link, "Invalid metric reading")
        ELSE
          APPEND (currentTime, bandwidth[link], delay[link],
            utilization[link], status[link])
            TO metricHistory[link]
        END IF
      ELSE
        GENERATE_ALERT(link, "Link failure detected")
      END IF
    END FOR
  END WHILE

```



```
// PSEUDOCODE CONTINUED
```

```
// Select the best valid path independently for each branch
FOR branch <- 1 TO BRANCH_COUNT DO
  bestPath <- NONE
  bestScore <- -1
  bestReason <- ""

  FOR EACH path IN candidatePaths[branch] DO
    (available, score, reason) <- EVALUATE_PATH(
      path, bandwidth, delay, utilization, status)

    IF available = TRUE AND score > bestScore THEN
      bestPath <- path
      bestScore <- score
      bestReason <- reason
    END IF
  END FOR

  IF bestPath = NONE THEN
    GENERATE_ALERT(branch, "No operational path to branch")
    DISPLAY "Branch ", branch, " is temporarily unreachable."
  ELSE
    currentUsable <- PATH_IS_USABLE(currentPath[branch],
      utilization, status)

    IF currentPath[branch] = NONE OR currentUsable = FALSE
      OR bestPath <> currentPath[branch] THEN
      oldPath <- currentPath[branch]
      ROUTE_TRAFFIC(branch, bestPath)
      currentPath[branch] <- bestPath
      GENERATE_ALERT(branch, "Path changed from ", oldPath,
        " to ", bestPath)
    END IF

    DISPLAY "Branch: ", branch, " | Selected path: ", bestPath
    DISPLAY "Quality score: ", ROUND(bestScore, 2),
      " | ", bestReason
  END IF
END FOR

SAVE_NETWORK_SUMMARY(currentTime, currentPath)
WAIT(MONITOR_INTERVAL)
END WHILE
END ALGORITHM
```

```
// PSEUDOCODE CONTINUED
```

```
FUNCTION EVALUATE_PATH(path, bandwidth, delay, utilization, status)
  bottleneckBandwidth <- INFINITY
  totalDelay <- 0
  totalUtilization <- 0
  hopCount <- LENGTH(path)

  IF hopCount = 0 THEN
    RETURN (FALSE, -1, "Empty path")
  END IF

  FOR EACH link IN path DO
    IF status[link] <> "UP" THEN
      RETURN (FALSE, -1, "Failed or invalid link")
    END IF

    IF utilization[link] > OVERLOAD_LIMIT THEN
      GENERATE_ALERT(link, "Overloaded link detected")
      RETURN (FALSE, -1, "Path contains overloaded link")
    END IF

    bottleneckBandwidth <- MIN(bottleneckBandwidth, bandwidth[link])
    totalDelay <- totalDelay + delay[link]
    totalUtilization <- totalUtilization + utilization[link]
  END FOR

  averageUtilization <- totalUtilization / hopCount
  bandwidthScore <- MIN(1, bottleneckBandwidth / REFERENCE_BANDWIDTH)
  delayScore <- MAX(0, 1 - (totalDelay / ACCEPTABLE_DELAY))
  utilizationScore <- MAX(0, 1 - (averageUtilization / 100))

  pathScore <- 100 * (0.50 * bandwidthScore
    + 0.30 * delayScore
    + 0.20 * utilizationScore)

  reason <- "BW=" + bottleneckBandwidth + " Mbps, delay="
    + totalDelay + " ms, utilization="
    + ROUND(averageUtilization, 2) + "%"
  RETURN (TRUE, pathScore, reason)
END FUNCTION

FUNCTION PATH_IS_USABLE(path, utilization, status)
  IF path = NONE THEN RETURN FALSE
  FOR EACH link IN path DO
    IF status[link] <> "UP" OR utilization[link] > 80 THEN
      RETURN FALSE
    END IF
  END FOR
  RETURN TRUE
END FUNCTION
```

Benefits of the Adaptive Algorithm

Network reliability: Failed, invalid, and overloaded links are excluded immediately, so traffic is not kept on an unsafe route.

Routing efficiency: The selected path balances usable bandwidth, total delay, and current utilization instead of selecting only the route with the fewest hops.

Fault tolerance: Candidate paths provide redundancy. When the current path fails or crosses the overload limit, an alternate valid path is selected automatically.

Adaptability: Metrics are recollected at fixed intervals, allowing the routing decision to follow daily changes in traffic and link quality.

Administrator awareness: Alerts, time-stamped histories, selected paths, and quality scores make every automatic decision visible and auditable.

Conclusion

The three algorithms use validation, arrays or lists, loops, conditions, functions, alerts, and complete summaries to measure performance and manage a changing network. Together, they calculate essential metrics, prevent congestion, and maintain reliable communication through adaptive routing.